

checkpointing

模型

确定性进程（Processus déterministes）：其状态可以通过初始状态和一系列事件完全定义。

非确定性进程（Processus non déterministes）：状态可能受到随机事件或外部条件的影响，无法完全预测。

基本原理

系统通过保存状态到可靠的存储介质上（例如磁盘或分布式存储），以便在出现故障时回滚至最近的「检查点」状态。

「检查点」后的状态可以通过回滚技术恢复，避免系统从头开始运行。

Panne

故障 => 从以前的状态恢复

单个进程的恢复点

方法：将上下文保存到可靠媒介中

成本优化：增量更新

非侵入式备份（备份时系统继续运行，但是可能导致保存不一致）

手段：Copy-on-write：当数据被修改时，创建被保护页面的副本。

Pre-copying：在恢复点创建之前复制所需数据。

一致性问题

一致性要求

凡因必有果

任何收到的消息都必须事先发送

恢复线前发送，恢复线前接收 ✓

恢复线前发送，恢复线后接收 ✓ -> 一致 cohérent

恢复线后发送，恢复线前接收 ✗ -> 不一致 incohérent

恢复线后发送，恢复线后接收 ✓

时间悖论（Paradoxe temporel）：

在分布式系统中，消息的发送和接收必须满足因果顺序。一旦撤销了消息发送，就必须撤销接收，然而接收可能已触发其他操作，从而引发更大的回滚。

这种情况类似于“时间悖论”，撤销一个事件可能导致整个事件链条逆转。

domino effect

多米诺效应：

回滚具有传播性，一个进程的回滚会逐步传播到依赖它的所有进程，最终可能导致整个系统状态被重置到一个很久以前的状态。

系统的可恢复性因此变得非常低效。

撤销广播 => 撤销接收

对外部世界的操作无法回滚

所以系统需要通过「验证（Validation）」或「提交（Commit）」来确保这些不可回滚操作的正确性。

无法“撤消”外部操作=>“验证”外部提交（输出提交）

稳定储存：是一种能够在发生各种类型的故障（如暂时性故障或永久性故障）时，依然保持数据完整性的存储机制。

解决方案

单一故障：远程储存（易失性内存）

暂时故障：本地磁盘

永久故障：分布式文件系统

基于恢复点的恢复技术

非协调保存点

每个进程独立创建保存点，保存点之间没有全局一致性。

故障恢复时，需要找到一条「覆盖线」（*ligne de recouvrement*），即所有进程最近的一致状态。

缺点：需要额外的计算和通信成本来协调保存点的一致性。

协调保存点

在创建保存点时，进程之间进行同步和协调，确保所有保存点形成一致的状态。

特点：所有进程的最后一个保存点构成覆盖线。

故障恢复时，从覆盖线中的最后一个保存点恢复进程。

优点：恢复更快，无需寻找一致状态。

缺点：创建保存点时的同步开销较大。

日志

保存事件日志，记录所有消息传递和状态变更。

故障发生时，重放日志（*rejouer le même scénario*），恢复到故障前的状态。

优点：精确恢复到故障前的状态。

缺点：需要较大的存储空间来记录日志。

非协调保存点：

天真的技术（Technique naïve）：

每次发送消息后进行本地保存（Sauvegarde locale après chaque émission）：

理论上（Théoriquement bon）：

没有孤立消息 => 无多米诺效应；

故障发生在发送和保存之间 => 回滚至前一个保存点；

内存开销较小（每个节点只有一个保存点）。

实际上并不理想（Pas optimal en pratique）：

对于消息发送频繁的节点来说，保存成本高昂；

对于消息发送少但接收/处理大量消息的节点，回滚成本高。

独立于应用程序的备份（Sauvegardes indépendantes de l'application）：

问题（Pb）：找到一致的覆盖线（ligne de recouvrement cohérente）：

构建因果关系的集中式图；

使用分布式恢复算法。

构建因果图的集中式方法：

每个进程本地维护一张图；

图表示直接依赖关系，连接本地保存点与其他进程的保存点。

恢复过程（Recouvrement）：

广播请求（Diffusion d'une requête）；

收集本地图，构建全局图；

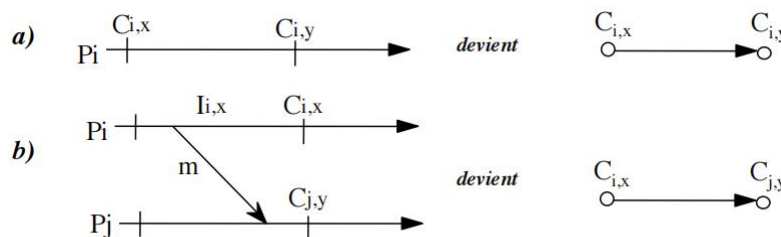
确定覆盖线；

重新开始执行。

基于图的两方法（Deux approches par graphes）：

回滚依赖图（Rollback dependency graph）：[Bhargava 88, Wang 95]；

保存点图（Checkpoint graph）：[Wang 93]。



1. a) 单进程依赖：

- P_i 的保存点 $C_{i,x}$ 和 $C_{i,y}$ 存在因果关系（即 $C_{i,y}$ 依赖于 $C_{i,x}$ ）。
- 恢复时，若要从 $C_{i,y}$ 开始恢复，必须先保证 $C_{i,x}$ 的状态正确。

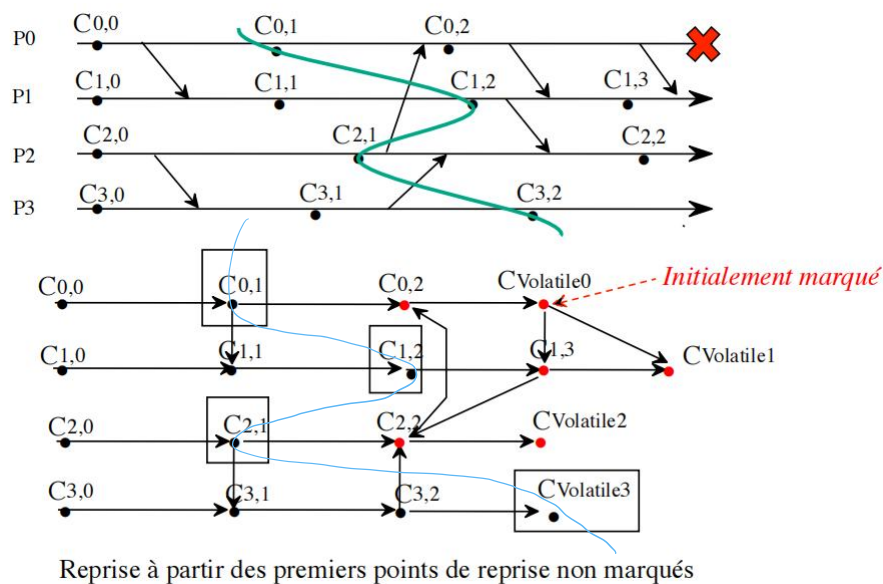
2. b) 多进程依赖：

- P_i 的保存点 $C_{i,x}$ 通过消息 m 影响了 P_j 的保存点 $C_{j,y}$ 。
- 因此， P_j 的 $C_{j,y}$ 依赖于 P_i 的 $C_{i,x}$ 。
- 若系统崩溃， P_j 必须等 P_i 回滚至 $C_{i,x}$ 后，才能正确恢复到 $C_{j,y}$ 。

关键机制

- 每条消息都会带一个标签（interval $\langle i,x \rangle$ ），说明它来自哪个检查点之后。
- 接收方在内存里更新依赖图（volatile checkpoint graph），这样就能记录消息和检查点的因果关系。
- 当产生新的检查点时，这个依赖图会写入稳定存储（stable storage），以便以后恢复时使用。

每条消息都有一个时间戳（如*<i>i,x</i>*），记录它的来源保存点。
 系统根据这些依赖关系，回溯保存点的因果关系，找到所有需要恢复的保存点集合。
 这些保存点构成「覆盖线」，保证了所有进程的一致恢复。



如果要回滚，依赖关系要求所有相关保存点也必须一起回滚，否则会导致数据不一致。
 从故障点开始分析，所有和故障点有依赖关系的 checkpointing 都不能用（如下半张图）

保存点图：记录了所有进程的保存点及其依赖关系。

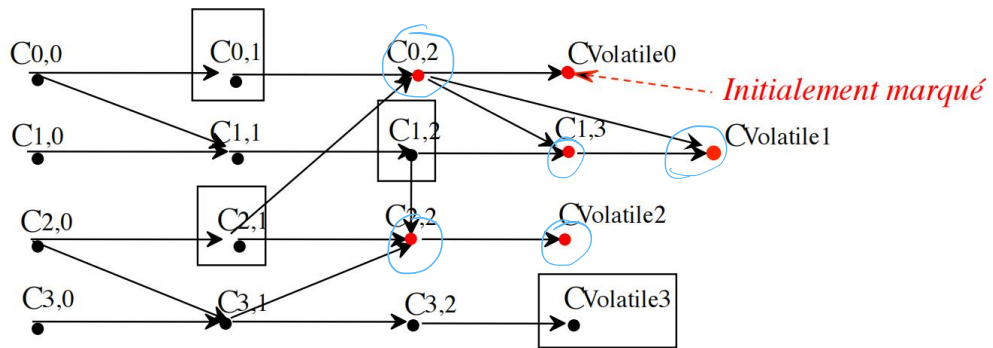
每次保存一个状态，就更新这个图，添加新的节点和边。

保存点图还能帮助清理已经过时或不再需要的保存点（比如那些没有依赖的点），减少存储开销。

RootSet 定义：

RootSet 是每个进程的最后一个保存点集合。

这些点是恢复的起点，但需要检查它们是否满足一致性。



计算覆盖线的步骤:

标记故障点:

从发生故障的保存点开始标记。

向前回溯:

从故障点沿依赖路径回溯, 标记所有受影响的保存点。

调整 RootSet:

如果 RootSet 中的某个点被标记, 那么替换为它的前驱点, 直到找到未受影响的保存点。

覆盖线确定:

最后的 RootSet 中的点就是覆盖线。

覆盖线的计算确保所有进程从一致的状态恢复, 并且不会遗漏任何依赖关系

分布式算法确定一致覆盖线 (在没有全局依赖图的情况下)

每个节点只知道自己的状态, 不知道其他节点的状态

难点: 复杂

成本高

强假设 (需要做出一些假设)

Juang-Venkatesan 算法:

该算法通过分布式消息传递和节点协作, 逐步标记受故障影响的保存点。

每个节点负责管理自己的依赖关系, 并通过与其他节点的通信确定一致恢复点。

特点:

它是一个基于因果关系追踪的算法;

通信成本较高, 但可以确保一致性。

Peterson-Kearns 算法:

强调通过最少的通信开销找到一致的恢复点。

特点: 相比 Juang-Venkatesan 算法更简化, 但依赖系统中较强的假设, 例如消息的可靠传递。

Juang-Venkatesan

假设

- 可靠信道和 FIFO
- 传输延迟未知但有限
- 无限本地内存
- 事件因果过程（分段确定性）

在本地，每个 P_i 进程维护

- 一个 S_i 状态计数器，该计数器在每次备份时递增
- 对于每个邻居 P_j ：发送计数器 $SENT_{ij}$ 和接收计数器 $RCVD_{ji}$
- 每个备份包括 S_i ，以及每个通道的 $SENT_{ij}$ 和 $RCVD_{ji}$

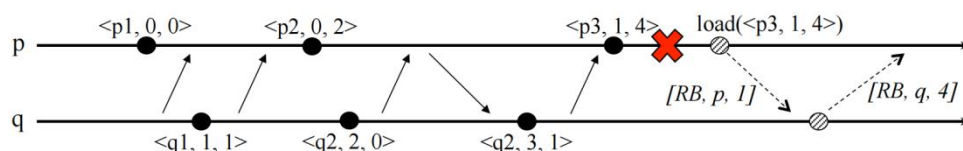
Juang-Venkatesan

Algorithme

```
Si ( $P_i$  défaillant) revenir au dernier état sauvegardé
Sinon créer nouveau  $S_i$  volatile

Répéter N-1 fois // N le nb de sites
  Pour tout site  $j \neq i$ 
    envoyer ( $\langle \text{ROLLBACK}, i, \text{SENT}_{ij}(S_i) \rangle$ )
  Répéter N-1 fois
    recevoir ( $\langle \text{ROLLBACK}, j, v \rangle$ )
    Si ( $\text{RCVD}_{ji}(S_i) > v$ ) // il y a des messages orphelins
      Trouver ( $S'_i$ ) le plus tardif tel que  $\text{RCVD}_{ji}(S'_i) \leq v$ 
       $S_i = S'_i$ 
  FinR
FinR
```

孤立消息：发出后，接收节点由于回滚而没有接收到/以及接收到但是被回滚掉的消息



1. 背景

这是一个 **非协调检查点 (uncoordinated checkpointing)** 的恢复算法。

- 每个进程 **独立保存** 自己的状态。
- 为了避免孤儿消息（收到但找不到发送方），需要在恢复时对消息收发做一致性检查。

2. 算法的核心思想

- 每个进程维护两个计数器：
 - SENT_{ij}**: 我发给邻居 j 的消息数
 - RCVD_{ji}**: 我从邻居 j 收到的消息数
- 每次 checkpoint, 会保存自己的状态 + 这两个计数器。
- 出故障时, 系统通过对比这些计数器, 来决定是否需要把某些进程回滚到更早的 checkpoint。

3. 算法步骤（翻译成人话）

假设进程 P_i 崩溃了, 要恢复:

- P_i 回到它最后一个保存的 checkpoint**
 - 如果 P_i 真的挂了, 就恢复到上一次 checkpoint。
 - 如果 P_i 还在, 就先创建一个“临时 checkpoint”。
- 通知其他进程**
 - P_i 给其他所有进程发一个消息:
`ROLLBACK, i, SENTij(Si)`
(意思是: “我在 checkpoint S_i 的时候, 发给你 j 的消息数是这么多”)
- 其他进程检查一致性**
 - 每个进程 j 收到这个消息后, 看看:
它记录的 **收到的消息数 RCVD_{ji}(S_j)** 是否大于 P_i 声称的 **SENT_{ij}(S_i)**。
如果大于 $\rightarrow$ 说明它“多收到了一些消息”, 也就是出现了 **孤儿消息**。
- 回滚到合适的 checkpoint**
 - 如果有孤儿消息, 进程 j 必须回滚到一个更早的 checkpoint S'_j ,
保证 **RCVD_{ji}(S_j) $\leq$ SENT_{ij}(S_i)** (也就是“收到的消息不能比 P_i 发的还多”)。
- 不断传播**
 - 这样回滚可能会牵连更多进程 (因为一旦 j 回滚, 它的邻居也可能出现不一致)。
 - 所以整个系统会逐步回滚, 直到所有进程的计数器都一致为止。

Peterson-Kearns

环上的矢量时钟

假设

- 具有未知但有限传输延迟的可靠信道
- 组织成环的过程（每个过程都知道它的继任者）

在本地，每个进程 P_i 维护一个矢量时钟，化身数在每次重叠时递增

Peterson-Kearns

Algorithme

```
Si ( $P_0$  défaillant)
    Revenir au dernier état sauvegardé  $S_0$ 
    Création d'un jeton  $\langle [incarnation++] \text{ (horloge de } S_0) \rangle$ 
    Propagation dans l'anneau

Réception de  $\langle [incarnation] \text{ (horloge de } P_i) \rangle$  sur  $P_j$ 
    Si ( $(P_j \text{ initiateur}) \ \&\& \ (\text{horloge}_j[j] \geq \text{horloge}_i[j])$ )
        FIN
    Si ( $\text{incarnation}_{\text{locale}} < \text{incarnation}$ )
         $\text{incarnation}_{\text{locale}} = \text{incarnation}$ 
    Si ( $\text{horloge}_j[j] < \text{horloge}_i[j]$ )
         $S_j = S_j$  telle que  $\text{horloge}_j[j] \geq \text{horloge}_i[j]$ 
        Propagation de  $\langle [incarnation] \text{ (horloge de } S_j) \rangle$ 
    Sinon
        Propagation de  $\langle [incarnation] \text{ (horloge de } S_i) \rangle$ 
```

1. 核心思路

- 用**向量时钟 (vector clock)** 来追踪事件因果关系。
- 所有进程按 **环形 (anneau/ring)** 排列（每个进程只需要知道下一个进程是谁）。
- 一旦某个进程崩溃，就用一个 **令牌 (token)** 带着故障信息和向量时钟，在环上依次传递，让大家一起对齐到一致的恢复线。

2. 算法假设

- 信道可靠，消息不会丢，而且是 FIFO 传输。
- 传输延迟未知但有界。
- 每个进程有无限内存，可以保存足够多的检查点。
- 进程是 *piece-wise deterministic*（除了消息，其他都是确定性的）。

3. 算法步骤（翻译成人话）

假设进程 P_0 崩溃：

1. P_0 回到它最后一个 checkpoint

- 加一个新的“incarnation number”（版本号，表示这是第几次恢复）。
- 创建一个令牌： $\langle \text{incarnation, horloge de } S_0 \rangle$ ，里面带着向量时钟信息。

2. 令牌在环上传播

- P_0 把令牌传给下一个进程 P_1 ， P_1 检查：
 - 如果自己的向量时钟太“新”（包含了故障前不该有的消息），它就必须回退到一个更早的 checkpoint，保证一致性。
 - 然后更新令牌里的信息，继续传下去。

3. 环路结束

- 令牌转一圈后回到 P_0 。
- 如果所有进程都调整完毕，就确定了一条全局一致的恢复线。

协调保存点

SaS 同步并停止 (sync-and-stop)

Point de reprise coordonnés : Sync-and-Stop (SaS)

Un coordinateur appelle une barrière de synchronisation pour geler l'application (arrêt des envois de message)

Algorithme :

1. Le coordinateur diffuse un message **chkptReq** à tous les processus
2. Réception **chkptReq** sur p :
p stoppe l'application (plus d'envoi) et renvoie **chkptReady** au coordinateur
3. Lorsque le coordinateur a reçu **chkptReady** de tous les processus, il diffuse un message **chkptDo** et sauvegarde son état
4. Réception de **chkptDo** sur p :
p sauvegarde son état et renvoie **chkptDone**
5. Lorsque le coordinateur a reçu tous les **chkptDone**, il diffuse **chkptCommit** pour débloquer les processus

算法步骤

1. 协调者广播检查点请求 (chkptReq)

- 协调者向所有节点广播一个消息 **chkptReq**，通知它们准备创建检查点。
- 所有节点收到此消息后，停止处理应用程序的任务（冻结消息的发送和接收）。

2. 节点发送准备状态 (chkptReady)

- 每个节点在收到 **chkptReq** 后：
 1. 暂停应用程序的执行（停止消息传递）。
 2. 返回一个消息 **chkptReady** 给协调者，表示它已准备好创建检查点。

3. 协调者确认所有节点准备就绪

- 当协调者收到来自所有节点的 **chkptReady** 消息后，表示整个系统已准备好进行检查点操作。
- 它广播一个消息 **chkptDo**，指示所有节点保存检查点。

4. 节点创建检查点并确认完成

- 每个节点在接收到 **chkptDo** 后：
 1. 保存当前状态到检查点。
 2. 返回一个消息 **chkptDone** 给协调者，表示检查点操作已完成。

5. 协调者释放所有节点

- 当协调者收到所有节点的 **chkptDone** 消息后：
 1. 它广播一个 **chkptCommit** 消息。
 2. 所有节点恢复应用程序的正常操作。

Algorithme non bloquant : Chandy-Lamport 85

- Algorithme “distributed snapshot” : sauvegarde cohérente de l'état global (état des processus + canaux de communication)
- Hypothèse : canaux FIFO
- Le coordinateur diffuse un *marker* et fait son point de reprise
- Réception du marker sur p sur le canal c:
 - Si p n'a pas encore sauvegardé son état :
 - Rediffusion du marker et sauvegarde d'état
- Tout message reçu sur le canal c entre la sauvegarde et la réception du marker sur c est sauvé (message en transit)

算法步骤

1. 标记消息传播:

- 发起者进程 (Initiator) 发送一个特殊的标记消息 `marker` , 标志快照的开始。
- 其他进程在收到标记消息后, 继续将标记消息沿通道传播。

2. 进程状态保存:

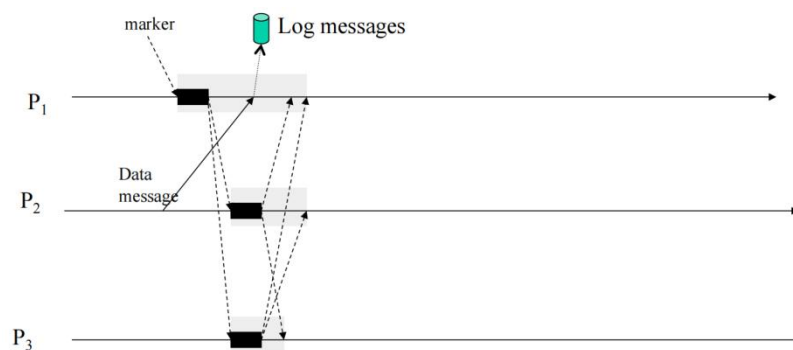
- 每个进程在第一次接收到标记消息时, 保存自己的本地状态。
- 如果已保存过状态, 则仅继续传播标记消息, 不再重复保存。

3. 通道状态记录:

- 在进程保存状态之后, 它开始记录从对应通道接收到的所有消息, 直到接收到来自该通道的 *marker* 标记消息为止。
- 这些消息被认为是传输中的消息。

4. 快照完成:

- 所有进程保存完状态, 并且所有通道的状态记录完成后, 整个系统的全局快照完成。

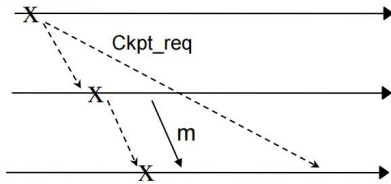


2. Lai-Yang (1987)

- 假设：信道 **不是** FIFO (可能乱序)。
- 机制：
 - Lai-Yang 改进了 Chandy-Lamport, 提出 **彩色消息 (colored messages)** 的方法：
 - 每个进程在开始快照时，把自己标记为“红色 (red)”。
 - 所有后续发出的消息也染色为 red。
 - 如果一个进程收到 red 消息，而它还没保存快照 → 它就立刻保存自己的状态。
 - 这样即使信道乱序，也能保证快照一致，因为“颜色”取代了 FIFO 假设。
- 图的右边：
 - Initiateur 发出 `ckpt_req`。
 - 普通消息 `m` 可能在 `ckpt_req` 之前或之后乱序到达。
 - 靠“颜色”区分哪些消息要算进快照里。

这是 Lai-Yang 算法，解决了非 FIFO 信道下的快照一致性问题。

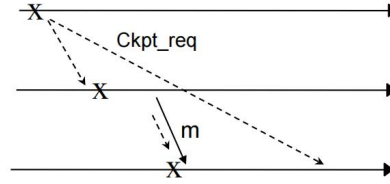
Initiateur



[Chandy-Lamport 85]

Hypothèse : canaux FIFO

Initiateur



[Lai-Yang 87]

Hypothèse : canaux non FIFO

Koo & Toueg 87

Messages estampillés – Chaque processus q conserve :

$Last_msg_q(p)$ - numéro du dernier message reçu de p depuis le dernier checkpoint

$First_msg_q(p)$ - numéro du premier message envoyé à p depuis le dernier checkpoint

Création d'un point de reprise PR

1. Notification aux processus dont on a reçu un message postérieur à PR-1
2. Test d'incohérence de chaque processus émetteur

Incohérence $\Rightarrow$ création d'un nouveau point de reprise par l'émetteur

Reprise

Annulation des envois postérieurs aux derniers points de reprise

最小同步协调的恢复点

Koo & Toueg 87

- **标记的消息**：每个进程 q 保存以下内容：
 - $Last_msg_q(p)$ ：自上一次检查点后，从进程 p 接收到的最后一条消息的编号。
 - $First_msg_q(p)$ ：自上一次检查点后，向进程 p 发送的第一条消息的编号。
- **创建恢复点 (PR) :**
 1. 通知那些在恢复点 PR-1 后接收到消息的进程。
 2. 检测每个消息发送方的**不一致性**。
 - 如果存在不一致性，则发送方创建一个新的恢复点。
- **恢复：**
 - 撤销所有在最后一个恢复点之后发送的消息。

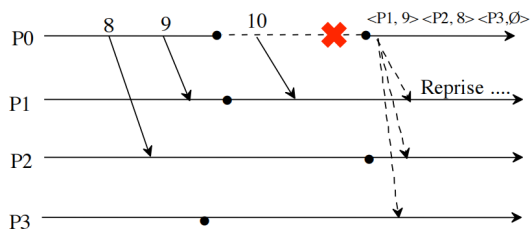
颜色规则 (核心四条, 背下来就会做题)

1. 初始全白：所有进程和消息初始都是 white。
2. 发送者变红：发送者 (initiator) 把自己标为 red，并从此只发送红消息 (red message)。
3. 第一次见红一轮回：任何进程第一次收到红消息时，立即：
 - 记录自己的本地状态；
 - 把自己标成 red，以后只发红消息；
 - 开始为每个人发消息时自一红发界前收到的白消息 (见下一条)。
4. 传递状态的判定 (或传递)：对某条入消息 q ，在该进程变红之后，直到在 q 上第一次收到红消息为止，这段时间从 q 收到的白消息，就是 q 的传递状态。一旦在 q 上见到第一条红消息， q 的传递状态封口。

直说：自一切变红之前，红一包围之后，我一直收到白消息，直到在我快照时已发出，但我还没收到红消息的传递消息，等到在这条消息上见到第一条红，说明该消息的白时代已被红消息前追上，可以关闭。

终止条件 (termination)

- 当每个进程都已经发过红并好每个人消息都见到了第一条红消息 (因而完成封口)，这轮快照就完成了，状态即可汇到一个收集器 (collector)。
- 若某消息这么久没有消息，为了让第一条红到达，完成封口，相关发送方需要主动发一条“空红消息” (null red) (工程上常叫 flush/heartbeat)。



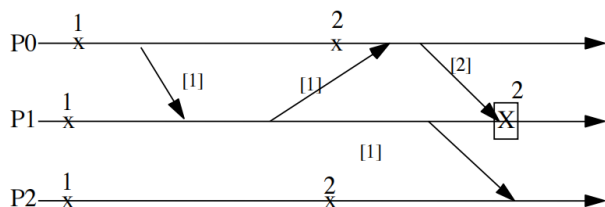
背景

在协调检查点里，最怕的就是 **多余回滚**：一个进程故障了，结果一大堆没问题的进程也被牵连回退（domino effect）。

Koo & Toueg 提出了一种 **最小化恢复** 的方法：只回退真正受影响的进程。

算法机制

1. 当 **q** 恢复时，它广播“在最后一次 checkpoint 之前，我发出的最后一条消息编号”（`last_msgq(p)`）。
 - 意思是：我告诉大家“我最多只认这些消息”。
2. 其他进程 **p** 收到后检查：
 - 如果 `last_msgq(p) < last_msgp(q)`，说明 q 收到的消息比它自己声称的还多（出现矛盾）。
 - 那么 p 必须回退，触发恢复。
3. **递归传播**：这样恢复请求会逐步传递，直到所有相关进程都退回到一致的状态。



背景

前面的协调 checkpoint 都需要显式的 **ckpt_req** 控制消息。

Briatico (1984)、Wu & Fuchs (1990) 提出：能不能利用 **应用消息本身** 来做隐式同步？

方法

- 每个进程维护一个 **计数器**（checkpoint counter）。
 - 每做一次 checkpoint，计数器 +1。
- 每条应用消息 m 都会带上发送方的计数器值。

协议

- 当接收方 P 收到消息 m 时，检查：
 - 如果 `m.compteur > compteur_local`，说明对方已经进入了更新的 checkpoint，自己落后了。
 - 那么 P 也必须立刻做 checkpoint，把自己的状态对齐。

Zigzag-Path (Xu & Netzer 93)

- Extension des chemins de causalité de Lamport
- Un *Zigzag chemin* (Z-path) de $C_{p,i}$ à $C_{q,j}$ est une séquence de messages $(m_1, m_2, \dots, m_l)$; $l \geq 1$, telle que:
 - m_1 est envoyé par p après $C_{p,i}$
 - Si m_k ($1 \leq k < l$) est reçu par r , alors m_{k+1} est envoyé par r dans le même intervalle (ou plus tard)
 m_{k+1} peut être envoyé après la réception de m_k
 - m_l est reçu par q avant $C_{q,j}$

Zigzag 路径 (Z-path) 的定义:

从 $C_{p,i}$ 到 $C_{q,j}$ 的 Zigzag 路径是一系列消息 $(m_1, m_2, \dots, m_l)$, 其中 $l \geq 1$, 满足以下条件:

1. 起点条件:

- 消息 m_1 是在检查点 $C_{p,i}$ 之后由进程 p 发送的。

2. 中间转发条件:

- 如果消息 m_k ($1 \leq k < l$) 被进程 r 接收, 则消息 m_{k+1} 是由进程 r 在相同时间间隔 (或更晚) 发送的。
- m_{k+1} 可以在接收 m_k 后立即发送, 也可以稍晚发送。

3. 终点条件:

- 最后一条消息 m_l 在检查点 $C_{q,j}$ 之前由进程 q 接收。

◆ Z-cycle 的“回到原点”

- 并不是说 回到同一个 checkpoint (比如 C1 又回到 C1) ,
- 而是说: 沿着消息接力走一圈, 回到同一个进程的某个 checkpoint。
- 典型情况:
 - 从进程 1 的 checkpoint 出发
 - 消息传到进程 3 的 checkpoint
 - 再经过一系列消息
 - 最后又回到进程 1 的 checkpoint

👉 这就构成了一个 Z-cycle。

两点
存在
因果
关系

通过日志的恢复

非协调的恢复点：系统不需要协调进程来设置统一的恢复点

基本原则：通过重放非确定性事件的日志来重建故障前的状态。

方法：记录所有的非确定性事件（日志记录）

假设：采用分段确定性（**piecewise deterministic, PWD**）模型。

执行被划分为一系列确定性区间。

每个区间由一个非确定性事件（如消息接收）启动。

悲观日志

同步日志记录：

每条接收到的消息都立即记录在日志中。

当进程恢复时，它从日志中读取消息并重放。

消息重传检测：

检测重传的消息（通过消息标记）。

拒绝来自自己恢复进程的重复消息。

优化：

硬件支持的稳定储存

没有专用硬件的时候可以利用发送方的内存

发送方记录 m 顺序

消息处理失败时，恢复的进程从发送方重传日志

乐观日志

异步日志记录：

日志数据暂时保存在 **RAM** 中，并定期同步到稳定存储中。

优点：

提高性能，因为不需要每次操作都同步到磁盘。

故障风险：

如果在日志同步到稳定存储之前发生故障，可能会丢失部分日志。

级联恢复（**Reprise en cascade**）：

故障的恢复可能导致其他相关进程也需要恢复。

优缺点：

优点：异步机制提高性能。

缺点：可能引发“多米诺效应”，导致更多进程参与恢复。

因果日志

因果关系图：

每个进程维护一个优先关系图，记录所有非确定性事件及其因果关系。

图的数据附加到消息中（即 **piggybacking**，消息捎带）。

用图指导恢复：

当某个进程需要恢复时，可以使用因果关系图来指导恢复，确保恢复后的状态保持一致。

	Non coordonnés	Coordonnés	Coordonnés implicites	Journalisation pessimiste	Journalisation optimiste	Journalisation causale
Hyp. PWD	Non	Non	Non	Oui	Oui	Oui
Surcoût Comm.	Faible	Aucun	Faible	Le plus élevé	Elevé	Elevé
Surcoût Sauvegarde	Faible	Le + élevé	Faible	Faible	Faible	Faible
Nb de pts de reprise	Plusieurs	1	1	1	Plusieurs	1
GC	Complexe	Simple	Simple	Simple	Complexe	Complexe
Reprise	Complexe	Simple	Simple	Simple	Complexe	Complexe
Effet domino	Possible	Impossible	Impossible	Impossible	Impossible	Impossible
Extension reprise	Illimitée	Dernier checkpoint	Dernier checkpoint	Minimum	Checkpoints précédents	Dernier checkpoint
"Output commit"	Impossible	Très lent	Très lent	Le plus rapide	Lent	Rapide

Tableau 1

策略	是否需要 PWD 假设	通信开销	存储开销	恢复点数量	GC 难度	恢复复杂度	多米诺效应	恢复范围	输出提交
非协调 (Non coordonnés)	否	低	低	多个	复杂	复杂	可能	无限	不可能
协调 (Coordonnés)	否	无	最高	1 个	简单	简单	不可能	最近检查点	很慢
隐式协调 (Coordonnés implicites)	否	低	低	1 个	简单	简单	不可能	最近检查点	很慢
悲观日志 (Journalisation pessimiste)	是	最高	低	1 个	简单	简单	不可能	最小范围	最快
乐观日志 (Journalisation optimiste)	是	高	低	多个	复杂	复杂	不可能	可能要回多个点	中等
因果日志 (Journalisation causale)	是	高	低	1 个	复杂	复杂	不可能	最近检查点	快

✳ 第一页：Tolérance aux fautes et large échelle（容错与大规模系统）

这页主要在讲：为什么很多传统的容错机制不适合在大规模系统（比如分布式集群、超大规模并行系统）中直接使用。

- **Éloignement（距离）**
节点之间相距很远，消息传输延迟很大 → 协议延迟变高。
(例如，跨洲的分布式系统，消息要几百毫秒，容错协议就很慢。)
- **Nombre de sites（节点数量多）**
节点越多，资源消耗越大（带宽、存储、CPU 开销）。
- **Dynacité（系统动态性）**
节点经常上下线，拓扑变化大。大部分容错策略是**静态的**（提前定死策略，或者由用户配置），不太适合动态环境。
- **Topologie（拓扑）**
大规模分布式系统容易**分区**（Partitioning），部分节点之间无法通信，导致协议失效。

👉 **总结：**
传统的容错方法更适合小规模、拓扑稳定的系统，但在大规模系统中会遇到严重性能和管理问题。

容错模型

- **Crash-recovery：**
大多数模型只考虑节点崩溃 → 重启恢复，这相对简单。
- **Byzantine faults（拜占庭错误）：**
越来越多系统（特别是 P2P、区块链）需要考虑节点可能是恶意的、发送错误消息。
解决方法：**BFT（Byzantine Fault Tolerance）库**，但非常昂贵。

✳ 第二页：Point de reprise pour les Grilles（网格计算中的恢复点）

这一页讲的是在 **Grid / 大规模集群** 系统中，如何分层做 checkpoint（检查点）。

- **层次化方法（Approche hiérarchique）**
比如 ProActive 系统里的做法（一个 PhD 论文提出的）：
 - **组级别的恢复：**
 - 每个 **cluster（集群）** 定义一个组（group）。
 - **组是恢复的最小单位。**也就是说，如果集群里某个节点坏了，就整个组一起恢复。
 - **组内协调（coordinated checkpointing within group）：**
 - 组内的 checkpoint 是协调好的。
 - **组间日志（inter-group message logging）：**
 - 不同组之间的消息会被记录，用日志的方式保证跨组恢复。

👉 **总结：**
在大规模系统中，用**分层的 checkpoint + 日志**来降低复杂度：

- 组内 → 协调 checkpoint
- 组间 → 日志保证一致性

自稳定

向“正确”行为收敛

和初始值独立 与初始值无关

Stabilization Time 稳定时间： 从开始到稳定的时间

分布式系统

信息的局部性（Locality of information）

时间的局部性（Locality of time）

=> 非确定性（Non-determinism）

配置（Configuration）：系统中所有组件局部状态的组合。例如，分布式网络中，每个节点的状态形成系统的整体配置。

执行（Execution）：系统组件局部执行的交织。由于分布式系统的异步性，不同组件的执行顺序可能不同，体现了系统的非确定性。

系统分类

传统系统/非自稳定系统：

从特定的初始配置开始，系统立即表现出正确的行为。

自稳定系统：

从任何初始配置开始，系统最终达到其行为正确的配置。

1974 年由 Dijkstra 定义

1984 年由 Lamport 倡导解决容错问题

（分布式）任务

安全性（Safety）：不应该发生的坏行为

活性（Liveness）：（最终）应该发生的好行为

错误：使系统偏离任务规范

故障是指通过改变系统组件的正确功能而破坏任务规范的操作。

故障的特性

类型（Type）：常见的故障类型包括：

Fail-stop：组件停止工作，但其他组件能够检测到停止的发生。

Crash：组件突然失效，可能无法被直接检测到。

Omission：组件未能按要求发送或接收信息。

Byzantine：组件表现出任意错误行为，包括发送冲突信息或恶意行为。

持续时间（Duration）：指故障的持续时间。它可能是短暂的（瞬时故障）或长期的（持久故障）。

1. Overflow control (溢出控制)

```
C
x = ((x = x + 2) > 254) ? 0 : x + 2;
```

- 重点：即便发生溢出/上界错乱，逻辑会把 x 拉回到一个可控区间，继续保持“每次+2”的偶数节奏。
- 不变式 (Invariant)：输出为偶数；当 x 走到边界时，重置到 0 以避免跑飞。

2. Parity check (奇偶校验，修正式)

```
C
x = (x % 2) ? x + 1 : x + 2;
```

- 重点：如果 x 不幸变成奇数，先 +1 纠正成偶数，再按偶数步进。
- 这就是一个典型的“自校正 (self-correction)”策略：任何“偏离偶数不变式”的状态都会被一步拉回。

3. Parity check — Reset (奇偶校验，复位式)

```
C
x = (x % 2) ? 0 : x + 2;
```

- 重点：发现错误（奇数）就直接复位到一个“已知好”的偶数 0。
- 牺牲连续性，换来更强更快的纠偏。

4. Left shift (左移输出)

```
C
printf("%d", x << 1);
x++;
```

- 重点：不修状态、改映射——输出时对 x 左移一位，哪怕 x 是奇是偶， $x \ll 1$ 永远是偶数。
- 这是一种把“正确性转移到”可观测行为“上的技巧：内部状态可乱，输出仍保持正确不变式。
- 对比前面三种“修状态”的方法，这种是“稳输出 (stable output)”的思路。

关键点：

- Locality of information：节点只知道邻居或自己的局部状态；
- Locality of time：不同节点操作的时间不同步；
- => 导致 non-determinism (非确定性)：不同调度顺序可能导致不同执行轨迹。

定义：

- Configuration (配置)：整个系统的全局状态 = 各节点局部状态的笛卡尔积。
- Execution (执行)：各个节点本地操作交错 (interleaving) 组成的全局运行。

→ 铺垫：在分布式环境里，我们根本没法保证所有节点一开始就在正确状态，因为信息和时间都“碎片化”。

检测率（Detection Rate）：表示系统检测故障的能力。有些故障（例如 Byzantine 故障）可能难以检测。

修复率（Correction Rate）：系统在检测到故障后，能够修复该故障的效率。

发生频率（Frequency）：故障的发生频率对系统设计至关重要。高频率故障可能需要更强的容错机制。

故障容错算法（Fault-tolerant Algorithm）：能够容忍一定类别故障的算法。

故障容错的分类：

Masking Fault Tolerance（掩蔽型容错）：

定义：同时保证系统的安全性（Safety）和活性（Liveness）。

特点：无论发生什么故障，系统始终保持正确的行为。

挑战：FLP 不可能性定理：在异步分布式系统中，无法在出现一个进程故障的情况下，完全保证一致性和活性。

应用难点：同时满足两个条件可能会导致算法过于复杂，甚至不可实现。

Fail-Safe Fault Tolerance（故障安全型容错）：

定义：仅保证安全性（Safety），但不保证活性（Liveness）。

特点：即使发生故障，系统的错误行为也不会违反安全性约束。但系统可能会进入停滞状态，无法继续运行。

示例：交通信号灯进入闪烁橙灯状态（避免冲突，但无法正常调度交通）。

Non-Masking Fault Tolerance（非掩蔽型容错）：

定义：仅保证活性（Liveness），但不保证安全性（Safety）。

特点：系统在发生故障后可能会短暂进入不安全的状态，但最终会恢复到正确状态。

示例：乐观算法：数据复制机制中，系统可能临时失去一致性，但最终会达到一致状态。

自稳定算法：最终可以保证安全性

最终 safety
liveness

Dijkstra 的自稳定算法

在环上传递令牌

在具有 4 个状态/进程的链上传递令牌

通讯方式

信息传递，最难写，但是消耗资源最少
共享寄存器，中等，消耗资源中等
共享内存，最好写，消耗资源也最多

1. 自稳定的前提是假设某些基本操作是原子的。
2. 即使代码形式看似“永远正确”，在字节码层次也可能被打断。
3. 因此，设计自稳定算法时，必须明确：哪些操作是原子的？哪些不是？

共享内存定义（Shared Memory）

描述：在共享内存模型中，所有节点可以通过共享的存储区域读取其他节点的状态，并更新自己的状态。

操作以原子性步骤进行：

读取所有邻居节点的状态。

基于读取的状态计算并更新自身状态。

关键点：

原子性（Atomicity）：确保读取和写入操作是不可分割的。

局部性（Locality）：每个节点只与其直接邻居交互。

定义（Guarded Command，受守卫命令）

格式：Guard → Action

解释：

Guard（守卫条件）：

一个基于邻居节点状态的逻辑谓词。

当条件为真时，表示当前节点可以执行相关操作。

Action（动作）：

当 Guard 条件为真时执行的具体操作，通常是更新自身状态。

意义：

Guarded Command 是分布式算法的基础机制，用于控制节点行为的触发条件。

调度（守护进程）

守护进程在可激活的处理器中选择将执行其操作的处理器。

守护进程可以被视为一个对手，其作用是阻止稳定

编程难度

分布式 > 同步

分布式 > 集中

资源消耗

同步 > 分布式

集中 > 分布式

编程难度

不公平 > 公平 > 有界

资源消耗

有界 > 公平 > 不公平

为什么把 daemon（守护进程/调度器）看作对手 adversary？

在分布式自稳定理论里：

- 节点都有一套局部规则（Guard → Action）。
- 但什么时候、哪个节点执行动作，完全由 daemon 决定。

1. 调度器很“刁钻”

- 它可能总是让某些节点执行，却长时间不调度另一些节点。
- 它可能在系统还没稳定前就不断“打乱节奏”，延缓收敛。
- 甚至，它可能安排动作的次序，让系统在很长时间内一直“乱跳”，迟迟不收敛。

2. 算法必须对最坏情况鲁棒

如果 daemon 很“善良”，那收敛就简单了；但自稳定理论的要求更强：

- 不管 daemon 怎么作恶（只要满足基本的公平性条件），系统都必须最终稳定。
- 这就好比算法要在 最糟糕的调度 下也能成功。

3. 对手视角的意义

- 把 daemon 看作 adversary（对手），有助于证明系统的鲁棒性。
- 我们设计算法时，就像在和 一个“捣乱鬼”博弈：
 - 它想阻止你稳定；
 - 但你的算法必须保证 无论它怎么安排调度，最终都能稳定。

小结

守护进程被看作对手，是因为它可以随意选择执行顺序和时机，潜在地阻止或延缓稳定。一个自稳定算法必须保证：即使在这种最坏调度下，也必然会收敛到合法状态。

1. Unfair（不公平调度）

- 调度器（daemon）可能永远不给某个进程/节点机会。
- 极端情况：有些节点完全“饿死”（starvation）。
- 对编程者来说最难保证正确性，因为你不能假设所有节点都会 eventually 被执行。

2. Fair（公平调度）

- 每个可执行的节点/进程最终都会得到执行的机会，但没有时间界限。
- 即：只要无限时间过去，大家都会跑到，但可能等很久。
- 常见在很多理论模型里，默认假设 fairness 来保证 活性（liveness）。

3. Bounded（有界公平调度）

- 更强的假设：如果某个节点/进程准备好执行，那么它会在有限步数内（bounded steps）被调度。
- 这避免了“无限等待”，保证了 更强的进展性。
- 代价是实现更复杂，调度器需要更多资源（跟踪每个任务的等待时间）。

对比总结

- 难度（红色箭头）：Unfair → Fair → Bounded，越往右越容易编程（因为假设更强，系统更可控）。
- 资源消耗（绿色箭头）：Bounded > Fair > Unfair，越往右调度器开销越大。

Fair Composition（公平组合），用于组合多个自稳定（self-stabilizing）算法，使它们协同工作。

Basic idea

基本概念：

- 将多个自稳定算法 $Al_1, Al_2, \dots, Al_k$ 组合起来，使得后面的算法可以复用前面算法的结果。
- 举例：算法 Al_{i+1} 无法检测到前面 $Al_1, Al_2, \dots, Al_i$ 是否已经稳定，但它的行为就像这些算法已经稳定了一样。
- **关键点**：通过复用，提升整体系统的复杂功能。

传递函数

3. 公平组合的定义

一个算法 AI 是 A_1 和 A_2 的 **公平组合 (fair composition)**，如果在 AI 中，每个进程会交替地执行 A_1 和 A_2 的动作。

也就是说，系统的 **调度器 (daemon)** 要保证两种算法的动作都有机会被执行，而不是只执行其中一个。

4. 定理

如果：

- A_1 是 **自稳定** 的，能解决任务 T_1 ；
- A_2 在给定 T_1 的前提下是 **自稳定** 的，能解决任务 T_2 ；

那么：

- **公平组合** A_1 和 A_2 后，整个系统在任务 T_2 上仍然是 **自稳定** 的。

换句话说，公平组合保证了自稳定性的 **可组合性**。

定义：

- 系统从一个配置 c_1 转移到下一个配置 c_2 ，再转移到 $c_3, c_4, \dots, c_i$ 。
- 每次转移都会使得某个函数 $FP(c)$ 的值不断下降，即 $FP(c_1) > FP(c_2) > FP(c_3) > \dots > FP(c_i)$ 。
- 最终 $FP(c_i)$ 达到一个稳定的边界值 bound。

用途：

- 用来证明系统的收敛性 (convergence)。
- 帮助计算系统从任意初始状态达到合法配置所需的步骤数。

强制系统走向正确

收敛阶梯（逐层转移）

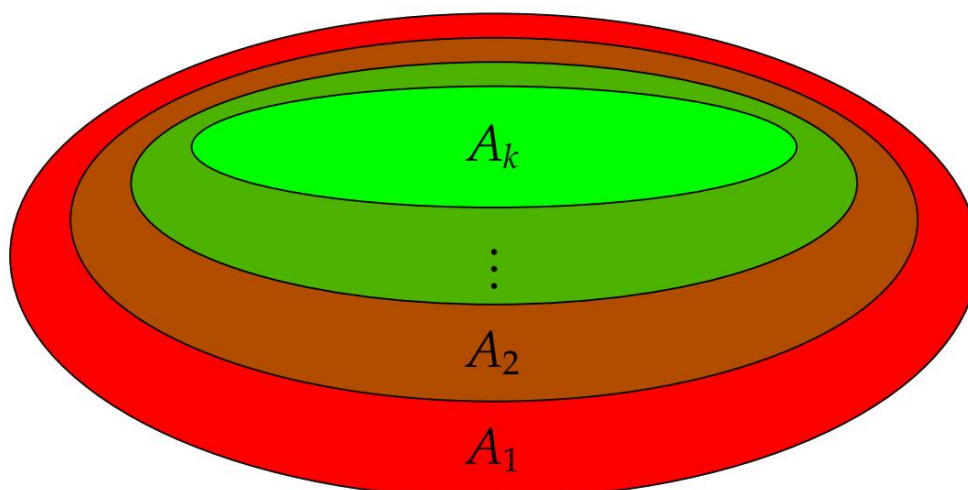
A_i 是谓词

A_k 是合法的

对于 1 和 k 之间的任何 i， A_{i+1} 是 A_i 的细化

系统的目标就是进入绿色的合法区域

从 A_1 一级级进入 A_k



自稳定

优点（Pros）

网络无需初始化（The network does not need to be initialized）：

自稳定算法可以在任何初始状态下运行，系统最终会自动达到合法状态。

故障处理简单（Fault handling is straightforward）：

如果检测到故障，只需识别并移除或重启有问题的组件，系统会再次自我恢复。

不依赖于故障的性质（Independent of the nature of the fault）：

无论故障的类型或原因是什么，自稳定系统都可以自动恢复。

不依赖于故障的程度（Independent of the extent of the fault）：

即使多个组件同时出错，自稳定特性仍然保证系统最终能够恢复到合法状态。

缺点（Cons）

时间无界（Unbounded stabilization time）：

虽然系统“最终”（Eventually）会稳定，但算法不保证稳定所需的时间上限。

节点无法判断是否已稳定（Nodes cannot determine stabilization state）：

系统中的节点在运行时无法确定系统是否已经稳定，只能依赖全局观察。

单点故障可能触发全局修正（Single failure may cause global correction）：

一个节点的故障可能导致整个网络触发校正动作，带来较大的开销。

故障假设（Faults must be transient）：

自稳定假设故障是暂时的，无法处理持久性故障（如硬件损坏或长期失效）。

1. 图的定义:

- $G = (V, E)$ 表示一个连通网络:
 - V : 由 n 个节点或进程组成的集合。
 - E : 由 m 条双向链路组成的集合。

2. 节点的邻居:

- N_p : 节点 p 的邻居节点集合, 即与 p 直接相连的节点。

3. 共享内存:

- 邻近节点之间可以共享内存, 用于交换信息。

分布式算法与同步

1. 计算步骤:

- 每个计算步骤中, 节点根据自己的变量及其邻居的变量决定是否启用 (enabled) 某个动作。

2. 系统分类:

- **同步系统 (Synchronous System) :**
 - 所有被启用的节点**同时**执行动作。
 - 强调全局的时间一致性, 通常需要全局时钟。
- **异步系统 (Asynchronous System) :**
 - 只有部分被启用的节点会执行动作。
 - 节点的动作由一个不公平的对手 (unfair adversary) 选择执行。
 - 更贴近实际分布式系统中的情况, 因为没有全局时钟。

逻辑时钟同步 (Logical Clock Synchronization)

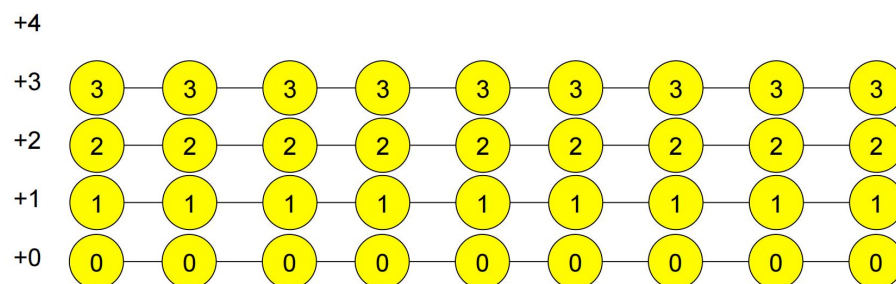
1. 每个节点维护一个逻辑时钟寄存器 r_p :

- 节点 p 的时钟值 r_p 负责记录当前节点的逻辑时钟状态。
- 时钟值通过递增规则更新, 即 $r_p := r_p + 1$ 。

2. 适用环境:

- 同步 (Synchronous): 所有节点可以在同一时间步执行。
- 异步 (Asynchronous): 节点在不同的时间步执行, 具体时间由系统的不公平选择机制决定。

同步系统: 强调统一的时间步。



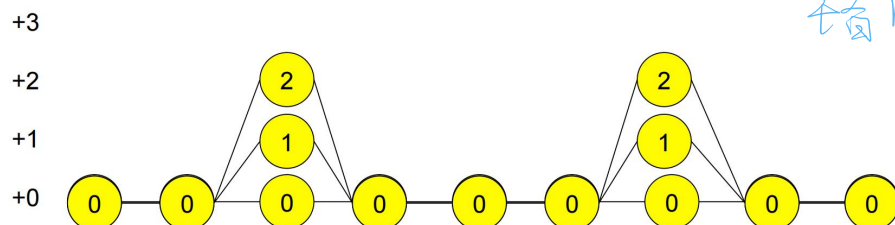
添加同步器

同步器的作用:

在异步环境中, 通过添加同步器来协调不同节点的逻辑时钟, 使得分布式系统中的逻辑时钟同步。

同步器确保节点之间的逻辑时钟按照一定顺序递增, 而不会因为异步操作导致失序。

异步系统: 强调局部节点间的协调和更新顺序。



错误子例

Global Synchronizer（全局同步器）（异步分布式系统）

在以下系统中：拥有唯一 ID 或者一个特定节点（如根节点、领导节点、主服务器等）。

波形算法，例如：

信息传播与反馈（PIF）

深度优先令牌循环

全局同步任务，例如：

（群体）互斥

领导者选举

系统重置

逻辑时钟同步

根树生成

带反馈的信息传播

单次传播的复杂度：

最佳情况： $O(D)$ ， D 是网络的直径。

在网络结构很好的情况下（比如直径较小）

最差情况： $O(N)$ ， N 是网络的节点总数。

在网络结构很差时（比如链式结构），需要遍历所有节点

Unison

每个节点在 $[0, \dots, K-1] = \mathbb{Z}_K$ 中维护一个相位时钟寄存器 r

安全性：相邻两个节点的相位时钟之间的间隙最多等于 $1 \pmod K$

无饥饿（活力）：每个相位时钟 r 无限频繁地增加 $1 \pmod K$

$(1 \pmod k)$ ，在 $k=1$ 的时候，结果为 0）

$K=2$ 的时候无法保证局部节点的逻辑顺序

当 $K=3$ 时，能够保证局部节点的逻辑顺序。

Lifting 机制

$p.R$ ：虚拟寄存器，记录节点 p 的时钟递增次数。

δ_0 ：虚拟寄存器的状态，通过提升机制（lifting）记录节点的全局增量。

γ_0 ：阶段时钟的状态，表示当前 $p.r \pmod K$ 的值。

提升机制就是把 虚拟时钟（ R ） 的信息映射到 实际时钟（ r ） 上，确保时钟值是有序的。

这里的 K 就是 时钟的模数，决定了时钟寄存器的取值范围。

解释：

- 每个节点的相位时钟 r 不是无限增长的整数，而是 循环计数（模 K ），
- 形式上：

$$r \in \{0, 1, 2, \dots, K-1\} = \mathbb{Z}_K$$

- 当 $r = K-1$ 再加 1 时，会回到 0。

举个例子：

- 如果 $K=4$ ，每个节点的时钟值只能是 $\{0, 1, 2, 3\}$ 。
- $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0 \rightarrow 1 \rightarrow \dots$
- 这就像一个 4 格的循环表。

为什么要引入 K ？

- 避免无界增长：时钟无限增长在分布式系统里不现实，寄存器/变量都是有限大小。
- 方便比较：用 $\pmod K$ 形式可以说“两个邻居的时钟最多差 1”，而不会出现无限差值。
- 保证循环同步：大家像一盏灯高一样轮流闪烁，始终保持节奏。

两个核心性质

1. Safety（安全性）

- 邻居之间的时钟差不能太大，最多相差 $1 \pmod K$ 。
- 换句话说：你不能快跑太多，你最多领先邻居一步。
- 这样保证大家的“节奏”不会乱套。

2. No starvation / Vivacity（不会饿死 / 活力）

- 每个节点的时钟 r 一定会不停地前进 $(+1 \pmod K)$ ，而不是卡住不动。
- 这样保证系统不会停摆，大家都能不断向前走。

兼容局部顺序：

假如两个节点有这样的顺序关系： $p.R \leq q.R$ （虚拟时钟上， p 的时钟比 q 小），那么在**实际时钟（ r ）**上，也会体现出来： $p.r \leq_i q.r$ 。

这样一来，虽然时钟值是循环的（比如 0, 1, 2），但通过提升机制，我们可以在所有节点之间定义一种全局的排序规则。

简单总结：**Lifting** 就是用“虚拟计数器”帮我们理清循环时钟中的全局顺序问题。

推广规则(在循环时钟中比较谁先谁后)

如果两个时钟值 a 和 b ，它们的差满足：

$$0 \leq b - a \bmod K \leq M$$

那么我们说 $a \leq_i b$ 。

M 决定了这个“范围”的大小， K 是时钟的循环范围。

$M \geq 1, K \geq 2M+1$

例 1: 比较 $4 \leq_i 1$

- 计算 $b - a \bmod K$:
 - $b = 1, a = 4$
 - $b - a = 1 - 4 = -3$
 - $-3 \bmod 5 = 2$ (因为 $-3 + 5 = 2$)
- 检查条件:
 - $0 \leq 2 \leq M(= 2)$
- 结论:
 - $4 \leq_i 1$ 成立。

例 2: 比较 $0 \leq_i 1$

- 计算 $b - a \bmod K$:
 - $b = 1, a = 0$
 - $b - a = 1 - 0 = 1$
 - $1 \bmod 5 = 1$ (因为 1 本来就在范围内)。
- 检查条件:
 - $0 \leq 1 \leq M(= 2)$
- 结论:
 - $0 \leq_i 1$ 成立。

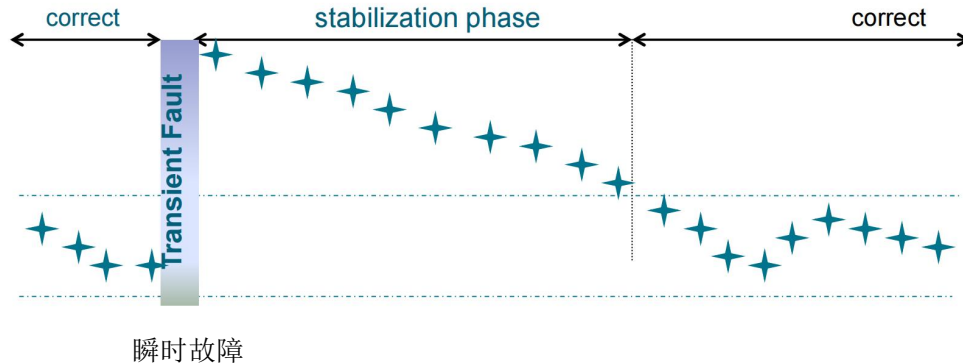
空间复杂度： $O(1)$

时间复杂度：

最好情况： $O(1)$

最差情况： $O(D)$ (延迟)

自稳定性



适当增加状态数量 (K) 是解决分布式系统死锁的一个重要方法。

系统配置的分类：

1. Γ : 寄存器的值可以是任意的。
2. Γ_1 : 两个相邻节点的寄存器值之间的差距小于或等于1 (这是一个局部约束)。
3. Γ_1^0 : 系统没有死锁 (存在一种"提升"机制, 可以兼容当前的系统配置)。

关键点：

- Γ_1 是 Γ 的子集, Γ_1^0 是 Γ_1 的子集。也就是说, 满足更多约束的系统更接近无死锁的状态。

条件 (定理) :

- 如果 $K > C_G$, 则系统可以从 Γ 的任意状态恢复到 Γ_1^0 (无死锁状态)。
- C_G (图的圈复杂度) :
 - 是图中最大的简单环路的长度 (一个环的"基准大小")。
 - 如果图是无环的, 则 $C_G = 2$ 。

环形复杂度 C_G 是图 G 中最大环的最小基数的大小。在一个网络图中, 环的复杂度是用于衡量多余边和闭环的一个重要特征。

关键结论: C_G 的大小决定了系统的同步条件。例如:

- $K > C_G$ 是确保环状网络中无死锁的重要条件。
- 对于更高 C_G 的结构 (如蜂巢), 系统的 K 值需要更高, 才能保证无死锁状态下的操作。

- C_G : 叫做图的 **cyclomatic characteristic** (环路秩/循环秩)。

它大概就是“图里面最大环的大小”。

- 如果图是棵树 (无环), 那 $C_G = 2$ 。
- 如果图里有一个很大的环, C_G 就等于那个环的长度。
- 只要你的时钟模数 K 比图中最大环的长度还大, 那么系统就不会死锁。
- 因为即便节点们错乱了, 通过同步, 它们也能收敛回去。

4. Reset 机制

如果节点检测到「自己和邻居时钟差太大」→ 触发 reset，把自己的时钟拉回到一个安全范围 $([-\alpha, \dots, 0])$ 。

这样系统就不会无限制乱下去，最终一定会收敛到稳定状态。

全局重置与局部重置：

全局重置：需要一个“根”节点或明确的 ID 系统。

局部重置：可以在匿名网络中工作，也就是没有 ID 标识的网络。

匿名性的动机：

在匿名网络中，使用少量的比特信息就可以区分大量的节点。

这种方法在简化基础设施的同时，也可以保障隐私。

匿名解决方案的优点：

不需要基础设施：

无需唯一的标识符分配，或者不需要中心化的控制节点。

不需要维护分布式的拓扑结构。

经济性：由于不需要额外的网络支持，可以降低成本。

隐私保护：用户身份信息可以得到更好的保护。

无需点对点路由：减少对复杂路由机制的需求。

非常适合传感器网络：因为这种场景中通常节点数量大且资源有限。

自稳定逻辑时钟同步的目标就是：

无论分布式系统一开始多乱，或者中途出错，只要 K 足够大、 α 设置合理，系统一定会自动恢复到所有节点时钟几乎同步的状态，而且保证既不会死锁，也不会让某个节点永远落后。

避免饥饿：

- 需要满足条件： $K > C_G$ ，其中 C_G 是图的循环复杂度 (cyclomatic characteristic)。

- 这确保了即使初始值是任意的，系统也可以避免饥饿。

安全性：

- 通过重置机制 (reset mechanism) 实现：寄存器的值被设定在区间 $[-\alpha, 0]$ 内。

- 条件： $\alpha > H_G + 1$ ，其中 H_G 是图的最大无弦环的长度。

SSSync 算法核心规则

1. 规则：

- AN (正常步骤)：所有正常节点都按照当前值 r_p 应用函数更新。
- AR (异常重置)：如果某节点未在尾部 (tail) 且非正常状态，则重置为 $-\alpha$ 。
- AC (初始化)：从初始状态出发，确保全局有序性， r_p 的更新遵循邻居节点的顺序。

2. 核心逻辑：

- 保证所有节点能够从任意状态最终达到同步，避免死锁或错误状态。

	# of states <i>状态数</i>	Stabilizing Time
Gouda, Couvreur, Francez, 1992	$O(n^2)$	$O(nd)$
Dolev, 2000	$O(n^2)$	$O(d)$
SSSync(K, α, M)		
$\alpha=0, K>M \cdot C_G, M>H_G-2$	$O(nd)$	$O(nd)$
$\alpha>H_G-2, M=1, K>C_G$	$O(n)$	$O(n)$
Tree networks	$O(1)$	$O(d)$

自稳定 ρ -Wavelets

- **内容:**
 - 当时钟寄存器 r 的值等于特定值（例如 0）时，触发一个特定事件“决定(DECIDE)”。
 - **定理:** 自稳定的阶段时钟同步协议提供了一种自稳定的 ρ -wavelet 机制。
- 如果 $\rho = \text{Diameter}(\text{Diam})$ ，上述协议可在匿名网络中实现“信息传播与反馈 (Propagation of Information with Feedback) ”。
- 一个集合 S 上的下界操作符 $\oplus$ 满足：
 - **结合性(Associative)**
 - **交换性(Commutative)**
 - **幂等性(Idempotent)**，即 $x \oplus x = x$ 。